

Mid-Semester Progress Report

Summer 2026– DSA 5900– 1 Credit Hour

Danny Nguyen

Faculty Supervisor: Dr. Matthew Beattie

Thesis Advisor: Dr. Charles Nicholson

Contents

List of Tables i

1 Introduction 1

2 Objectives 2

 2.1 Technical Project Objectives 2

 2.2 Individual Learning Objectives 2

3 Data 3

 3.1 Ingestion 3

 3.2 Exploration 3

 3.3 Preparation 4

4 Methodology 5

 4.1 Techniques 5

 4.1.1 Algorithmic Formulation and Modeling 5

 4.1.2 Train-Test Splits and Validation Strategy 5

 4.2 Process Validation 6

5 Results 7

6 Deliverables 8

7 References 9

List of Tables

1 Introduction

In the modern media landscape, the velocity and volume at which information spreads make manual verification insufficient, leaving professional journalists and publishers working under severe time constraints. This problem has catalyzed a growing interest in fact-checking systems capable of parsing arbitrary text, retrieving evidence, and predicting the veracity of assertions. [2] [3] However, contemporary semantic pipelines frequently rely on generative large language models (LLMs) to output structured configurations like JSON, introducing critical flaws: they are highly prone to hallucination, fragile under slight parsing variances, and computationally expensive at scale. [4] [5] [8]

To overcome these limitations, this practicum aims to define a low-compute, symbolic framework to extract the underlying predicate logic from synthetic natural language reasoning examples using frozen transformer activations. By deploying a compact, open-weights model—Meta’s Llama 3.2-3B—this practicum extracts high-dimensional activation vectors from the model’s intermediate residual stream layers. Rather than relying on text generation, these numerical parameters will feed into shallow, linear classification probes trained to deterministically isolate atomic assertions and identify propositional relationships (e.g., implication, negation, and conjunction) directly from the embeddings within a model’s latent space. [1] [6]

2 Objectives

The overarching goal of this practicum is to develop a robust, parameter-level extraction pipeline that bypasses generative LLM bottlenecks to map natural language into verifiable logical graphs. The specific project and individual objectives are outlined below:

2.1 Technical Project Objectives

- **Latent Space Probing Architecture:** Design and train low-compute, linear classification heads (probes) to extract propositional claims and logical relationships directly from the intermediate residual stream of Meta's Llama 3.2-3B model.
- **Symbolic Pipeline Integration:** Construct a deterministic pipeline that translates extracted latent representations into discrete symbolic tokens without relying on stochastic text-generation or JSON parsing.
- **Graph-Based Fact Verification:** Implement a visualization and verification layer that structures the extracted symbols into a formal logical graph, for human interpretation of the underlying logical structures of natural language.
- **Performance Optimization:** Evaluate the classifier framework to achieve high factual and structural precision while reducing inference-time computational overhead compared to baseline autoregressive generative prompting.

2.2 Individual Learning Objectives

- **Mechanistic Interpretability Mastery:** Gain deep, practical expertise in analyzing the internal geometry of transformers, specifically concerning layer-wise activation tracking, representation dynamics, and internal knowledge localization.
- **Advanced Representation Engineering:** Master state-of-the-art diagnostic probing and activation patching methodologies using specialized interpretability toolkits.
- **Symbolic Architecture Development:** Develop a rigorous engineering workflow for bridging machine representations (vector embeddings) with classical symbolic paradigms (graph structures).

3 Data

3.1 Ingestion

The dataset selected for this project is **LogicNLI** [7], an open-source diagnostic benchmark hosted by the Hugging Face hub. The dataset in JSON was retrieved directly from the authors' GitHub repository. ^[1] There have been no obstacles in acquiring the dataset, as it is publicly available and well-documented thanks to the work of the authors. The dataset is stored locally in a structured directory within the project repository, ensuring easy access for preprocessing and analysis.

3.2 Exploration

Initial exploratory data analysis revealed a highly uniform, balanced, and synthetically controlled dataset structure across its 20,000 entries. The target `labels` field contains a near-perfect balanced split across its classification categories (Entailment, Contradiction, Self-Contradiction, and Neutral) eliminating the risk of majority-class bias during probe calibration.

A structural analysis of the text fields highlights the specific syntactic attributes of the dataset:

- **Facts Array:** This field contains flat, atomic assertions of absolute truth (e.g., ["Alice is in the garden.", "Bob is hungry."]). These strings exhibit zero logical connectives, maintaining a short average string length of 4 to 6 tokens.
- **Rules Array:** This field holds the propositional conditional formulas (e.g., ["If Alice is in the garden, then Alice is reading."] or structures introducing logical equivalence and conjunction ("and")). These strings demonstrate higher token complexities and specific syntactic anchors that dictate the relationship edges.
- **Statements and Labels:** The `statements` field presents the test query, which correlates cleanly to an explicit truth value defined by the formal deduction of the combined rules and facts.

Exploratory token length histograms further confirmed tight distribution peaks, ensuring that the model's intermediate hidden layers will not suffer from sequence padding distortions during batch processing.

^[1]<https://github.com/omnilabNLP/LogicNLI/tree/main>

3.3 Preparation

Because the linear probes require distinct feature-target pairs to map out Llama's internal vector space, substantial preprocessing and feature engineering were executed to transform the raw dataset into a token-level sequence classification task. No missing data or null fields were detected due to the synthetic nature of LogicNLI, bypassing the need for standard statistical imputation.

The data preparation workflow consists of two critical pipeline stages:

1. **Rule-Based Target Transformation and Fact-Rule Matching:** A custom semantic parsing script was built to map the structural elements of the environment. Within the `rules` array, regular expressions and token-dependency parsing isolate the relational connectives, explicitly identifying whether a rule represents an implication ($\rightarrow$) or a conjunction ($\wedge$). Concurrently, a lookup matrix maps these rules directly back to the atomic strings inside the `facts` array. This enables the framework to identify exactly which baseline facts are being operated on by any given rule, providing clear structural targets for the relational edge probes.
2. **Contradiction-Driven Negation Curation:** Because explicit negation ($\neg$) is frequently represented implicitly across text pairs rather than through simple keyword matching, a target-generation strategy was implemented leveraging the dataset's logical symmetries. The pipeline filters entries where the target `label` is explicitly marked as a "contradiction". By aligning the queried `statement` against its corresponding asset in the `facts` array under this specific label, the pipeline mathematically isolates the exact token span undergoing negation. This contradiction-driven mapping provides a high-precision, curated subset of explicit positive-to-negative vector transformations.
3. **Token Span Realignment:** Because character indices shift once strings are converted into transformer sub-tokens, the Hugging Face tokenizer was configured with the `return_offsets_mapping=True` parameter. This maps the character spans generated during the rule-parsing and contradiction-alignment steps directly onto the sequence tensor positions of Meta's Llama 3.2-3B model. This preparation ensures that when forward hooks intercept the hidden states of layers 8 through 16, the mean-pooling functions grab the precise vector slices corresponding to the isolated logical components.

4 Methodology

4.1 Techniques

To extract and verify the logical structures embedded within Meta’s Llama 3.2-3B model, this practicum implements a supervised learning paradigm using diagnostic probing—specifically, the linear classification approach described by Alain, et al. [1] Rather than treating the transformer as an end-to-end text generation system, this approach treats the internal intermediate activations as a high-dimensional feature space from which structured semantic primitives can be decoded.

4.1.1 Algorithmic Formulation and Modeling

The foundational modeling technique utilizes shallow linear classification probes optimized via standard Logistic Regression with L_2 regularization. Mathematically, given a dataset of hidden state activation vectors $X \in \mathbb{R}^d$ extracted from a specific intermediate transformer layer l (where $d = 2048$ for Llama 3.2-3B), the probe seeks to learn a weight matrix W and bias vector b to predict the discrete logical class Y (e.g., term boundaries or connective relations):

$$\hat{Y} = \sigma(W \cdot X + b)$$

where σ represents the softmax function.

4.1.2 Train-Test Splits and Validation Strategy

The experimental validation follows a rigorous cross-validation protocol using the pre-partitioned LogicNLI splits (**train**, **dev**, and **test**):

- **Calibration Split:** The probes are trained using activation vectors extracted strictly from the **train** split.
- **Model Selection and Layer Auditing (Dev Split):** The validation framework uses the **dev** split to audit the model layer-by-layer. Probes will be independently trained and evaluated across layers 8 through 16 to map the trajectory of logical synthesis. The layer yielding the highest F1-score on the dev set will be selected as the optimal extraction layer.
- **Generalization Verification (Test Split):** Final model performance is reported strictly using vectors from the unseen **test** split. Out-of-sample generalization will be quantified using Precision, Recall, and F1-score relative to the ground-truth target structures.

4.2 Process Validation

The selection of a parameter-level probing methodology over standard autoregressive token generation was heavily shaped by preliminary research consultations with my thesis advisor. The overarching academic goal was to ensure this project delivered a high-quality, meaningful contribution to mechanistic interpretability by focusing on two core criteria: drastically increasing the ease of human verifiability in arbitrary text and optimizing performance under strict, low-compute constraints.

During the scoping phase, discussions with my advisor focused on the inherent limitations of using standard generative workflows for truth and fact-checking tasks. Forcing an LLM to output structural text or a natural language explanation often introduces stochastic hallucinations and abstract reasoning errors that a human reviewer cannot easily audit without manual fact-checking [3]. To ensure a higher standard of human verifiability, the choice to deploy diagnostic linear probes directly on the model's internal residual stream was validated. By extracting the raw latent geometries and mapping them directly to mathematical nodes and directed edges, we eliminate the black-box nature of text generation. This methodology allows an observer to easily verify conclusions based purely on the explicit, visualizable logical consistency of the statements made in the text.

Furthermore, this technique directly addresses the research requirement for compute efficiency. Rather than relying on massive, closed-source models or computationally expensive fine-tuning loops, this approach utilizes a compact 3B-parameter model and shallow classification heads. This design ensures that the entire extraction and verification architecture can perform with high structural precision while operating entirely within local, consumer-grade hardware constraints (such as an RTX 4070 Ti). This validation process guarantees that the methodology yields a reproducible, low-overhead framework that successfully bridges deep learning vector spaces with verifiable symbolic graph networks.

5 Results

TBD. I hope I have enough time try single-layer and multi-layer probes, and a couple different open models to see how results change by model size.

6 Deliverables

TBD

7 References

- [1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2018.
- [2] Fengxiang Cheng, Haoxuan Li, Fenrong Liu, Robert van Rooij, Kun Zhang, and Zhouchen Lin. Empowering llms with logical reasoning: A comprehensive survey, 2025.
- [3] Zhijiang Guo, Michael Schlichtkrull, and Andreas Vlachos. A survey on automated fact-checking, 2022.
- [4] Zhijing Jin, Abhinav Lalwani, Tejas Vaidhya, Xiaoyu Shen, Yiwen Ding, Zhiheng Lyu, Mrinmaya Sachan, Rada Mihalcea, and Bernhard Schölkopf. Logical fallacy detection, 2022.
- [5] Abhinav Lalwani, Tasha Kim, Lovish Chopra, Christopher Hahn, Zhijing Jin, and Mrinmaya Sachan. Autoformalizing natural language to first-order logic: A case study in logical fallacy detection, 2025.
- [6] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets, 2024.
- [7] Jidong Tian, Yitian Li, Wenqing Chen, Liqiang Xiao, Hao He, and Yaohui Jin. Diagnosing the first-order logical reasoning ability through LogicNLI. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3738–3747, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics.
- [8] Zhun Yang, Adam Ishay, and Joohyung Lee. Coupling large language models with logic programming for robust and general reasoning from text, 2023.